The assert line is doing the work: an anchor-leak ablation of EvaLoop

Shreyas Visweshwaran · CSC 591/791 Trustworthy AI for Software Engineering · Assignment 4 Fork:
https://github.com/FirefoxSRV/EvaLooop (branch `anchor-ablation`)

1. Extension and research question

EvaLoop measures robustness by feeding a model its own output: generate code from a description, test it, summarize the code, and let that summary become the next description. Three measurements on the provided data package suggested to me that the loop does less work than it appears to. Across all 107 leaderboard models I measured Spearman(pass@1, ASL) = **0.945** (R^2 = 0.81), so ASL is nearly rank-equivalent with one-shot accuracy; **91%** of loops passing cycle 1 never fail within ten cycles; and the median model has only **26** mid-loop failures out of 378.

One string never degrades. `CodeSummarizationTask.postprocess` appends the **final assert line of the original prompt** to every summary, forever. It exists for a real reason, since a summary does not fix the function's name and the tests could not otherwise call the code, but it carries more than a name: one concrete input with its **ground-truth expected output**, re-injected from outside the loop every cycle, in a framework whose premise is that model output is the only source of perturbation. Models exploit it: **33.4%** of all 165,630 recorded cycles contain an echoed module-level assertion (99.5% for gpt-5, 0% for gemini-2.0-flash).

> ***RQ.*** *How much of EvaLoop's measured robustness is the model sustaining its own specification, and how much is a re-injected ground-truth anchor holding the task still?*

I expected removing the oracle to lower ASL and raise mid-loop failures (H1). Hypotheses were fixed in `PREREGISTRATION.md` before any run.

2. The framework and my changes

The pipeline. `EvaLoopEvaluator` loads the 378 MBPP+ tasks, seeding each with the benchmark's docstring prompt. Per cycle, `BatchEvaluationCycle` runs the configured tasks through one interface, `create_prompt → model → extract_output → postprocess`. The generation task renders `"Generate Python code for the following task: …"` and extracts the first triple-backtick block. That code runs against MBPP+'s base and extended EvalPlus suites via `untrusted_check`, passing only if both pass. The summarization task then prompts for a one-sentence summary of that code, and its `postprocess` wraps the reply as a docstring with the assert line re-appended, which becomes the next cycle's description. A failing task stops; survivors continue to ten cycles. `successful_cycles` counts cycles before the first failure, and `analysis/metrics.py` turns that vector into ASL = $\Sigma_i\, n_i \cdot i^2/(T \cdot M)$, whose i^2 weighting pays compound credit for sustained cycles.

What I changed. The anchor was the only thing I needed to vary, and the `dev` branch already routes it through `postprocess`, so the extension is almost entirely new files: `evaloop/anchors/` (a registry mirroring `tasks.factory`, plus three strategies), two new task types, a Gemini backend, a disk-backed call cache, a sandboxed batch tester, and an evaluator subclass. **I changed five existing source files, +47/−8 lines.** The one substantive edit is in `evaluation/cycle.py`: upstream's `postprocess` context carried only the cycle-start input, and a model-derived anchor needs the code that was *just produced*, so I made the context also carry `task_input` (in both the batch and sequential paths; existing tasks ignore it). The rest are lazy-import moves in `models/factory.py` and `core/evaluator.py` so the generation stage needs neither torch nor evalplus, plus two registration lines and a docstring. I reused `analysis/metrics.py` and `analysis/similarity.py` rather than reimplementing the metrics or the judge.

Arm	Anchor	Carries
A0 `assert`	`assert find_length("11000010001") == 6`	name, arity, inputs, **expected output**. This is upstream unchanged.
A1 `masked_assert`	`assert find_length('11000010001') == ...`	name, arity, inputs; oracle masked by AST
A2 `signature`	`def find_length(s):`	name and parameter names, parsed from the model's own code each cycle

A1−A0 isolates the oracle, and A2−A1 isolates the remaining ground-truth-derived argument values. A2 is the only arm where nothing but model output crosses the boundary. For **signature stability**, the problem the assignment requires any Option B design to solve, I parse the code that just *passed* and emit the `def` line for the dataset's `entry_point`. Since that code passed the tests, a callable function with the right name and arity exists by construction. Because cycle 1 is byte-identical across arms and the call cache is shared per model, **pass@1 is equal across arms by construction**, so ΔASL is attributable to the anchor alone.

One confound had to be controlled. A1's anchor ends `== ...`, so a model that echoes it raises `AssertionError` and fails for a formatting reason, and an echoed assertion is part of the code summarized next, which could restate the oracle in prose. I therefore strip module-level `assert` statements from generated code before testing *and* before summarization, in **every** arm including the baseline.

Validation before any claim. With `anchor: assert` the new task reproduces **all 165,630** recorded cycle transitions in the data package byte-for-byte, so the baseline arm is provably the published loop. `analysis/metrics.py` reproduces the leaderboard for **56/60** models (the rest are the multi-run models its README documents). Re-executing 120 stored solutions through my sandbox reproduces **117/120** recorded verdicts, and assert-stripping changed **0 of 40** verdicts where it applied, so the rule is immaterial to outcomes. The framework's own `ResultAnalyzer` returns my ASL to ten decimal places. 254 tests pass.

3. Experimental setup

I ran the ablation at two scales. The **main experiment** is all **362 eligible MBPP+ tasks**, three arms each, on `gemma-4-26b-a4b-it` and `gemma-4-31b-it`. Eligible means the 362 of 378 tasks whose assert states a comparison; the other 16 assert bare truthiness and admit no

masked anchor, so A1 is undefined on them. There is no sampling, and the task set reproduces the 60-model leaderboard ranking at Spearman **ρ = 0.9966**.

A **cross-family check** adds `gemini-3.5-flash-lite` on a 150-task stratified subset (seed 20251013, five quantile strata by mean sustained cycles across the 60 package models, ρ = 0.937). It is capped at 500 requests per day, far short of the full set, so I kept it at the smaller scale rather than lose the only non-Gemma model I could reach.

The Gemini 2.5 series, the only models with baselines in the data package, now returns `404 … no longer available to new users`, so no evaluated model has a published baseline and **the A0 arm is the baseline**, which is what Option B requires anyway. Greedy (`temperature=0`, `top_p=1`), `max_output_tokens=1024`, `seed=20251013`, thinking pinned to `minimal`, max 10 cycles, one run per configuration.

Reproduction. Part of my first full-scale run was lost, so on Nov 1 I re-ran all six configurations from an empty cache. `gemma-4-26b` is **byte-identical to the original on all 2,799 cycles**. `gemma-4-31b` is not: the API now returns different code for **41 of 362 tasks at cycle 1**, so the served model changed. I report its rerun for all three arms, since mixing old and new arms would break the shared cycle 1; the original A0 is kept in `results_full/`. The 20 coded failures are `gemma-4-26b` loops, unchanged.

Untrusted code is executed only inside a container with `--network none`, a non-root user, dropped capabilities, a read-only repo mount and memory/CPU/pid caps; the test image holds no model SDK and no API key. The container is required rather than advisory: EvalPlus's `reliability_guard` sets `RLIMIT_AS`, which macOS rejects.

Cost. All runs were on the free tier: **$0.00 billed**. Rerun from an empty cache, the six full-scale runs needed **4,659 API calls** and 726,025 tokens, against **27,773 requests served from cache: 85.6% of everything the loop asked for was a byte-identical repeat it had already seen.** The two cross-family runs added 427 calls.

LLM use beyond the models under evaluation. I used an LLM as a classifier for the failure coding in Section 4, and I report it here as the assignment requires.

Model. Claude Opus 4, accessed through Claude Code. *Input per example.* Four artifacts drawn from the run records: the original MBPP+ task, the summary that fed the failing cycle, the code that passed the previous cycle, and the code that failed. The arm label was withheld, so the classification was blind to the condition under test. *Inference settings.* No per-example decoding parameters were set; this ran inside an interactive assistant session rather than as a metered API call, so temperature, token limits and seed were not controlled and the pass is not reproducible in the way the model runs are. *Conversion to a decision.* Exactly one primary code per failure, drawn from the eight-code codebook fixed in `PREREGISTRATION.md`, applying the rule that C1 covers the loss of a detail of the same operation and C2 a shift to a different coherent operation. C6 was excluded from the classifier's scope because it requires cross-arm information; `scripts/score_coding.py` derives it from sustained-cycle data instead. *Cost.* No separately metered per-example charge.

Two limitations follow and I would rather state them than let a reader infer them. There is **one coder, not two**, so the Cohen's kappa the codebook anticipated does not exist and the distribution in Section 4 has no inter-rater check behind it. And the 20 coded failures are close to the entire population of 24 mid-loop failures available, so this is nearer a census than a sample, which removes sampling error but also any claim to independence between the sample and the result.

The boundary-similarity judge (`scripts/judge_similarity.py`) reuses the framework's byte-identical prompt from the data package but I did **not** run it, so no number in this report depends on it. Had I run it, its scores would not have been comparable to the published `gpt-4-turbo-2024-04-09` ones.

4. Results and failure analysis

model	arm	pass@1	ASL	ASL_bias	mid-fail	surv. 10	ΔASL [95% CI]	Wilcoxon
gemma-4-26b	A0 `assert`	0.771	7.440	9.654	10	269	n/a	n/a
	A1 `masked_assert`	0.771	6.978	9.053	27	252	**−0.463 [−0.71, −0.22]**	**p = 2.9e−4**
	A2 `signature`	0.771	6.786	8.804	34	245	**−0.655 [−0.96, −0.36]**	**p = 4.7e−5**
gemma-4-31b	A0 `assert`	0.773	7.407	9.577	12	268	n/a	n/a
	A1 `masked_assert`	0.773	7.053	9.118	25	255	**−0.355 [−0.60, −0.14]**	**p = 5.8e−3**
	A2 `signature`	0.773	7.000	9.050	27	253	**−0.408 [−0.68, −0.16]**	**p = 2.4e−3**

(362 tasks each; paired bootstrap, 10k task resamples; two-sided Wilcoxon signed-rank on per-task sustained cycles. pass@1 is identical across arms by construction, since cycle 1 is a shared cached generation.)

The effect is real. Both models fall monotonically A0 → A1 → A2, and **all four comparisons are significant**: ΔASL −0.655 (p = 4.7e−5) and −0.408 (p = 2.4e−3) for the full ablation, and −0.463 (p = 2.9e−4) and −0.355 (p = 5.8e−3) for removing the oracle alone. Mid-loop failures roughly triple on `gemma-4-26b` (10 → 34) and double on `gemma-4-31b` (12 → 27).

On my earlier 150-task subset the full ablation gave `gemma-4-26b` −0.526 (p = 0.020); the full set **grew** it to −0.655 rather than shrinking it. **H1 is supported, at a magnitude still below the >1.0 loop I pre-registered**: the leak is load-bearing for a substantial minority of tasks, not for robustness in general.

The A1 arm now separates too: removing *only* the expected output, keeping the concrete arguments, costs `gemma-4-26b` 0.463 ASL, about 70% of the full ablation, and `gemma-4-31b` 0.355, about 87%. Most of what the anchor contributes is the answer, not the argument values.

Does the ranking change? Yes, for the full-scale pair: `gemma-4-26b` leads under A0 by 0.033 ASL, well inside the CIs, but `gemma-4-31b` leads under A1 and A2 by 0.075 and 0.214, because it loses less when the oracle is withdrawn. On the 150 tasks all three models share, the scores sit within 0.13 ASL under A0, and under A2 the top two swap by only 0.008 (ρ = 0.5, n = 3), which I do not read as

meaningful. H4 predicted the leak masks differences between models; the evidence is mixed (the Gemma pair's ASL_bias gap widens 0.08 → 0.25, the three-model sd narrows 0.123 → 0.109), so I count it as neither supported nor refuted.

What the extension reveals that the standard loop does not. Two structural findings, neither of which ASL exposes.

The sustained-cycle distribution is almost perfectly bimodal. Under A0, 83 tasks fail at cycle 1 and 269 survive all ten; **of the 362, one task fails anywhere between cycles 4 and 9.** The ablation's entire effect lands in the first two cycles: tasks sustaining exactly one cycle go 6 → 18 → 28 on `gemma-4-26b` and 11 → 23 → 23 on `gemma-4-31b`. The oracle matters at the first handoff from the model's own description, and essentially nowhere after it.

The loop reaches a fixed point almost immediately. Of the 279 tasks that get past cycle 1 under A0, **269 reach a repeated (code, summary) state**, at median **cycle 3**: the model regenerates identical code, which yields an identical summary, indefinitely. That is most of what the **85.6% cache-hit rate** measures, and it is why six full-scale runs cost 4,659 API calls against 32,432 requests. Cycles 4 to 10 re-derive a fixed point while ASL's i^2 weighting pays quadratic credit for them.

Manual inspection. I coded 20 mid-loop failures blind against the pre-registered codebook. They were drawn from the 150-task runs, so the counts below describe that frame, not the 362-task runs. The sampler shuffles the rows and withholds the arm labels in a separate key file, because knowing that a failure came from an ablated arm would make it trivial to code as oracle dependence. I therefore did not code C6 at all; I derive it afterwards from the run data.

arm	C1 detail loss	C2 operation drift	C4 signature	C5 extraction	C7 plus-only	n
A0 `assert`	1	0	1	1	1	4
A1 `masked_assert`	4	3	1	0	0	8
A2 `signature`	4	4	0	0	0	8

The modes differ sharply. The baseline failures I read were mechanical and idiosyncratic: one nested code fence that truncated mid-function, one `(m, n)` parameter inversion, one extended-test edge case, one garbled recursion. The ablated failures are **almost entirely specification degradation**, 8 of 8 and 8 of 8 in C1 plus C2.

Independent of how the mechanisms are coded, the cross-arm arithmetic is fixed: **11 of the 20 sampled failures are oracle-dependent**, meaning the same task sustained strictly longer under A0, and **10 of those went from surviving all ten cycles to failing at cycle 1**. That count is derived from sustained-cycle data, not from a judgement call.

Reading the cases, I found the example was doing *disambiguation* rather than supplying an answer to copy: it resolved element ordering (Mbpp/462), the return container (set vs list, Mbpp/473), rotation direction (Mbpp/586), whether "pairs of tuples" means pairs *of* tuples or the pair *within* one (Mbpp/120, Mbpp/409), and the treatment of zero digits (Mbpp/745). These models **never echo the assert (0.0% of cycles)**, so the leak cannot be a copyable crutch. That refutes my pre-registered H5 and replaces it with a better account.

A loop the extension broke. Mbpp/226: *"remove the characters which have odd index values"*, anchored `assert odd_values_string('abcdef') == 'ace'`. Under A0 it survives ten cycles. The summary becomes *"return a string containing only the characters at odd-indexed positions"*, which is coherent but **inverted**. Under A0 the expected output `'ace'` contradicts that reading and the model implements the original anyway; under both ablated arms it fails at cycle 1 with `return string[1::2]` → `'bdf'`. The description was always ambiguous; the oracle was resolving it.

A loop it did not. Mbpp/3: *"identify non-prime numbers"*, anchored `assert is_not_prime(2) == False`. Ten cycles under all three anchors; its summary stabilizes at cycle 1 and never moves, because the description fully determines the function. **That is the pattern: the anchor props up precisely the under-specified tasks, and ASL does not distinguish them from the well-specified ones.**

5. Limitations and implications

Threats. (i) **Test-suite strength.** A loop fails only when the suites catch the drift, so MBPP+'s tests bound this in both directions: weak tests hide real degradation, making sustained-cycle counts an upper bound on fidelity, while the extended EvalPlus inputs can end a loop for reasons unrelated to the anchor. One failure I coded is exactly that, a correct rotation that lost its empty-list guard. Because every arm runs identical suites on identical tasks, this moves all arms together and does not explain the between-arm differences, which is the claim being made. (ii) **Two models carry the full-scale ablation, and both are Gemma.** The third family is present only on the 150-task subset and only for A0 and A2, so the cross-family claim is the weakest thing here; the within-model paired comparisons, which are the evidence for H1, do not depend on it. (iii) The arms still differ on a minority of tasks, 20 to 36 out of 362, so each test rests on those discordant cases rather than on the whole set. That is far better than the 11 to 16 the subset gave, which is why all four comparisons separate, but it means the effect sizes rest on a few dozen tasks. (iv) MBPP is from 2021 and these models sit at 0.77 pass@1, so contamination is plausible, though it would inflate all arms equally. (v) The 16 tasks whose assert cannot be masked are excluded by design, so the result speaks to comparison-style asserts only. (vi) A2 changes information content *and* surface form; A1 partly separates them. (vii) Single run per configuration. (viii) One primary coder grading their own experiment, on the smaller frame; blinding and a sealed second coder mitigate but do not remove this. (ix) `gemma-4-31b` changed between my runs; hosted models are not fixed artifacts, so robustness scores should pin versions.

Implications. EvaLoop's ASL is a sound idea undermined by two design details. The loop converges to a fixed point by cycle 3, so ten cycles mostly re-derive the same strings while i^2 pays compound credit, which is why ASL tracks pass@1 at $\rho = 0.945$. And the anchor that keeps the harness runnable also re-injects ground truth, which disproportionately rescues under-specified tasks. Both are fixable without abandoning the metric: derive the anchor from the model's own output (A2 costs nothing extra and is strictly more faithful to the self-consistency premise), stop at the first repeated state rather than a fixed budget, and report the sustained-cycle histogram rather than a single i^2-weighted scalar. More broadly, a self-consistency benchmark should account for every channel through which the benchmark's own answers can re-enter the loop; here one line of scaffolding, present for a legitimate engineering reason, was quietly supplying the disambiguation that the measurement was supposed to be testing.